Concorrenza: mutua esclusione e Sincronizzazione

Il presente documento fornisce un'analisi approfondita dei principi di concorrenza, mutua esclusione e sincronizzazione nella gestione dei processi e dei thread all'interno dei sistemi operativi. La trattazione sintetizza i concetti fondamentali, le difficoltà intrinseche e i meccanismi di controllo necessari per garantire l'integrità del sistema.

Sintesi Esecutiva

La progettazione dei sistemi operativi moderni è intrinsecamente legata alla gestione della concorrenza, che si manifesta in contesti di multiprogrammazione, multiprocessamento e processamento distribuito. Il cuore della sfida risiede nel coordinare l'accesso a risorse condivise per evitare condizioni di errore non deterministiche, note come **Race Condition** .I punti chiave includono:

- **Gestione delle Risorse:** Il sistema operativo deve tracciare i processi, allocare/deallocare risorse e proteggere i dati dalle interferenze.
- **mutua esclusione:** È il requisito fondamentale che garantisce che un solo processo alla volta acceda a una sezione critica.
- **Meccanismi di Controllo:** Per risolvere i conflitti vengono impiegati supporti hardware (istruzioni atomiche) e costrutti software (semafori, mutex, monitor).
- **Problematiche Critiche:** Se non gestita correttamente, la concorrenza può portare a **Deadlock** (stallo), **Livelock** o **Starvation** (mancanza di risorse per un processo specifico).

Fondamenti e Terminologia della Concorrenza

La concorrenza nasce in tre contesti principali:

1. **Applicazioni Multiple:** Inventate per condividere il tempo di elaborazione tra più applicazioni attive.
2. **Applicazioni Strutturate:** Estensione del design modulare e della programmazione strutturata.
3. **Struttura del Sistema Operativo:** Gli stessi sistemi operativi sono implementati come insiemi di processi o thread.

Glossario Tecnico

Termine	Definizione
Operazione Atomica	Funzione o sequenza di istruzioni indivisibile, nessun altro processo può vederne lo stato intermedio o interromperla.
Sezione Critica	Porzione di codice che richiede l'accesso a risorse condivise e non deve essere eseguita contemporaneamente da più processi.
mutua esclusione	Requisito che impedisce a un processo di entrare in una sezione critica se un altro vi è già presente.
Race Condition	Situazione in cui più thread/processi leggono e scrivono dati condivisi e il risultato finale dipende dai tempi relativi di esecuzione.
Deadlock	Due o più processi sono bloccati perché ognuno attende un'azione dall'altro.
Livelock	Due o più processi cambiano continuamente stato in risposta l'uno all'altro senza compiere alcun lavoro utile.
Starvation	Un processo pronto per l'esecuzione viene ignorato indefinitamente dallo scheduler.

Principi e Difficoltà della Concorrenza

In un sistema uniprocessore, l'esecuzione dei processi è interfogliata (**interleaving**), mentre nei sistemi multiprocessore è sovrapposta (**overlapping**). Entrambi i modelli presentano le medesime problematiche: la velocità relativa di esecuzione dei processi non è predicibile, dipendendo dalle attività degli altri processi, dalla gestione delle interruzioni e dalle politiche di scheduling del sistema operativo.

Difficoltà Principali

- **Condivisione di risorse globali:** L'accesso non regolamentato a variabili o risorse comuni può corrompere lo stato dei dati.
- **Ottimizzazione dell'allocazione:** Per il sistema operativo è complesso gestire l'assegnazione delle risorse in modo ottimale.
- **Debugging:** Risulta difficile individuare errori di programmazione poiché i risultati non sono deterministici né facilmente riproducibili.

Responsabilità del Sistema Operativo

Per gestire la concorrenza, il sistema operativo deve:

1. Tenere traccia dei vari processi attivi.
2. Allocare e deallocare risorse per ogni processo.
3. Proteggere i dati e le risorse fisiche di ogni processo da interferenze esterne.
4. Garantire che l'output dei processi sia indipendente dalla velocità di elaborazione.

Interazione tra Processi

Il grado di consapevolezza che i processi hanno l'uno dell'altro influenza il tipo di relazione e i potenziali problemi di controllo.

Grado di Consapevolezza	*Relazione*	*Influenza tra Processi*	*Problemi di Controllo*
Inconsapevoli	*Competizione*	*Risultati indipendenti; il timing può essere influenzato.*	*Mutua esclusione, Deadlock (risorse rinnovabili), Starvation.*
Indirettamente consapevoli *(es. oggetti condivisi)*	*Cooperazione tramite condivisione*	*I risultati possono dipendere da informazioni altrui; coerenza dei dati.*	*Mutua esclusione, Deadlock, Starvation, Coerenza dei dati.*
Direttamente consapevoli *(comunicazione)*	*Cooperazione tramite comunicazione*	*I risultati dipendono da informazioni altrui.*	*Deadlock (risorse consumabili), Starvation.*

mutua esclusione: Requisiti e Supporto

Requisiti Fondamentali

La mutua esclusione deve essere applicata rigorosamente seguendo questi criteri:

- Un processo che si arresta nella sua sezione non critica non deve interferire con gli altri.
- Non devono verificarsi Deadlock o Starvation.
- L'accesso alla sezione critica non deve essere negato se questa è libera.
- Non si devono fare assunzioni sulla velocità dei processi o sul loro numero.
- Un processo deve rimanere nella sezione critica solo per un tempo finito.

Supporto Hardware

- **Disabilitazione delle Interruzioni:** Utilizzabile solo in sistemi uniprocessore. Garantisce la mutua esclusione ma degrada l'efficienza e non funziona su architetture multiprocessore.

- **Istruzione Compare&Swap (Atomic Exchange):** Esegue atomicamente il confronto tra un valore in memoria e un valore di test; se uguali, avviene lo scambio con un nuovo valore. Disponibile in architetture x86, IA64, SPARC e IBM.
- **Istruzione Exchange:** Scambia il contenuto di un registro con quello di una locazione di memoria. Implementata in Pentium e Itanium.

Vantaggi del supporto hardware: Applicabile a qualsiasi numero di processi (sia uniprocessore che multiprocessore) e semplice da verificare.

Svantaggi: Utilizza il **busy-waiting** (consumo di tempo CPU durante l'attesa), rischio di starvation e deadlock.

Meccanismi di Sincronizzazione

Semafori

Un semaforo è una variabile intera utilizzata per la segnalazione tra processi. Sono definite solo tre operazioni atomiche:

1. **Inizializzazione:** Con un valore intero non negativo.
2. **semWait:** Decrementa il valore. Se il valore diventa negativo, il processo viene bloccato.
3. **semSignal:** Incrementa il valore. Se il valore è minore o uguale a zero, un processo bloccato viene rimosso dalla coda e reso pronto.

- **Semafori Forti:** Garantiscono l'ordine FIFO (First-In-First-Out) per il rilascio dei processi dalla coda (evitano la starvation).
- **Semafori Deboli:** L'ordine di rimozione dalla coda non è specificato.
- **Semafori Binari:** Possono assumere solo i valori 0 e 1.

Altri Meccanismi

- **Mutex:** Simile al semaforo binario, ma il processo che blocca il mutex deve essere lo stesso a sbloccarlo.
- **Monitor:** Costrutto del linguaggio di programmazione che incapsula variabili e procedure. Solo un processo alla volta può accedere al monitor.
- **Condition Variables:** Utilizzate per bloccare un processo finché una determinata condizione non diventa vera.
- **Event Flags:** Bit di memoria usati per la sincronizzazione (supportano logiche AND/OR).
- **Mailboxes/Messages:** Mezzi per scambiare informazioni e sincronizzarsi.
- **Spinlocks:** Meccanismo di mutua esclusione in cui un processo attende in un ciclo infinito finché la variabile di blocco non diventa disponibile.

Implementazione POSIX

Nello standard POSIX, i semafori sono gestiti tramite il tipo sem_t. Le funzioni principali includono:

- sem_init(sem_t *sem, int pshared, unsigned int value): pshared è 0 se condiviso tra thread, 1 tra processi.
- sem_wait(&sem): Per l'operazione di attesa.
- sem_post(&sem): Per l'operazione di segnalazione (incremento).